

MongoDB Commands

R.Radhakrishna

2520090063

The screenshot displays the OneCompiler MongoDB Online Editor interface. The left pane shows a JavaScript script for interacting with a MongoDB database. The script includes commands for finding documents, inserting a new document, inserting multiple documents, and finding documents based on specific criteria like roll number, name, department, and marks. The right pane shows the output of the script, which is a JSON array of documents. The output includes the details of the inserted document and the documents found by the various queries.

```
14 })
15 db.students.find()
16 db.students.find().forEach(printjson)
17 db.students.insertOne({
18   rollNo: 111,
19   name: "Vijay",
20   age: 21,
21   department: "CSE",
22   marks: 86,
23   city: "Hyderabad"
24 })
25 db.students.countDocuments()
26 db.students.insertMany([
27   { rollNo: 112, name: "Pooja", age: 20, department: "IT", marks: 84 },
28   { rollNo: 113, name: "Ramesh", age: 22, department: "ECE", marks: 79 }
29 ])
30 db.students.countDocuments()
31 db.students.find()
32 db.students.findOne()
33 db.students.findOne({ rollNo: 101 })
34 db.students.find({ name: "John" })
35 db.students.countDocuments()
36 db.students.countDocuments({ department: "CSE" })
37 db.students.find({ marks: { $gt: 80 } })
38 db.students.find({ marks: { $gte: 80 } })
39 db.students.find({ marks: { $lt: 80 } })
40
```

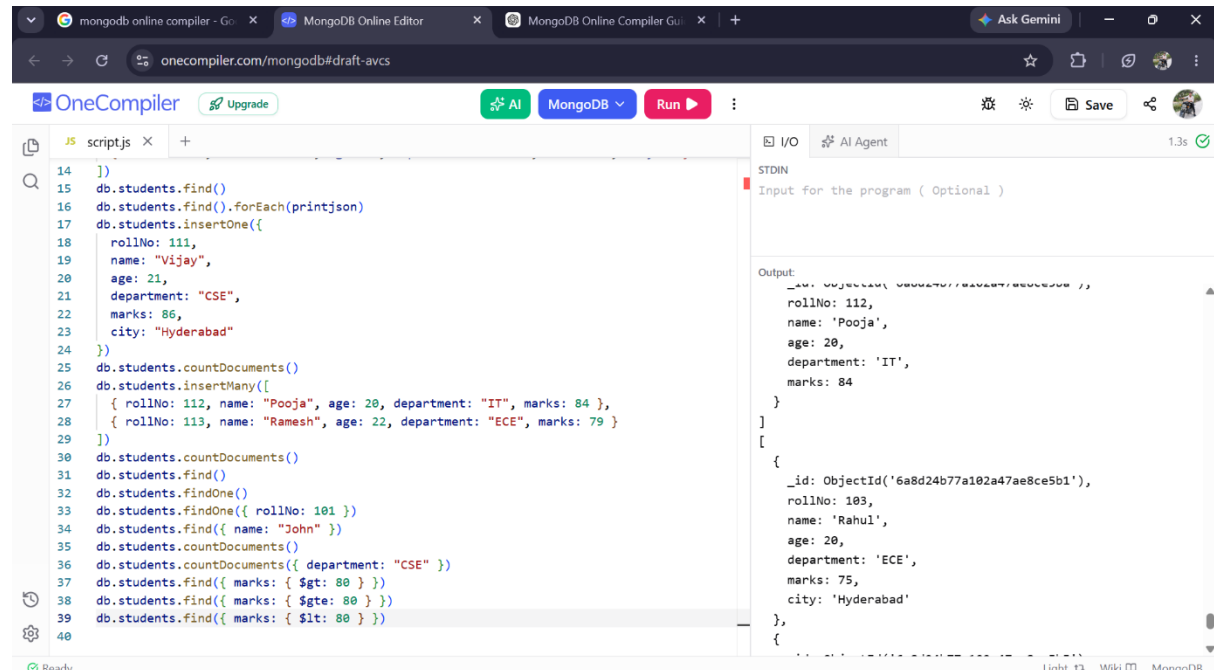
Output:

```
{
  "_id": ObjectId('6a8d24b77a102a47ae8ce5b5'),
  "rollNo": 107,
  "name": 'Ravi',
  "age": 23,
  "department": 'ECE',
  "marks": 68,
  "city": 'Nellore'
},
{
  "_id": ObjectId('6a8d24b77a102a47ae8ce5bb'),
  "rollNo": 113,
  "name": 'Ramesh',
  "age": 22,
  "department": 'ECE',
  "marks": 79
}
```

MongoDB Commands

R.Radhakrishna

2520090063

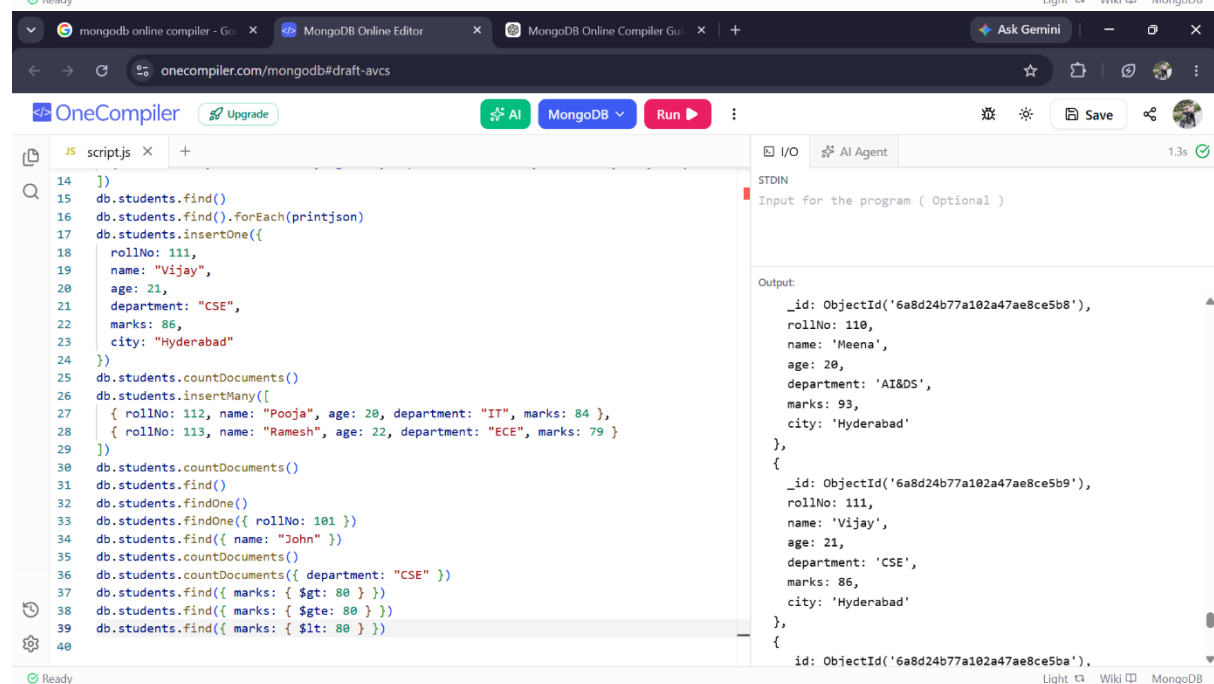


The screenshot shows the OneCompiler MongoDB Online Editor interface. The left pane contains a JavaScript script (script.js) with 40 lines of code. The right pane shows the output of the script, which is a JSON array of student documents. The script performs the following operations:

- Inserts a single document: `{ rollNo: 111, name: "Vijay", age: 21, department: "CSE", marks: 86, city: "Hyderabad" }`
- Counts the number of documents in the `students` collection.
- Inserts multiple documents: `{ rollNo: 112, name: "Pooja", age: 20, department: "IT", marks: 84 }, { rollNo: 113, name: "Ramesh", age: 22, department: "ECE", marks: 79 }`
- Counts the number of documents in the `students` collection.
- Finds all documents in the `students` collection.
- Finds a document by `rollNo: 101`.
- Finds a document by `name: "John"`.
- Counts the number of documents in the `students` collection.
- Counts the number of documents in the `students` collection where `department: "CSE"`.
- Finds documents where `marks: { $gt: 80 }`.
- Finds documents where `marks: { $gte: 80 }`.
- Finds documents where `marks: { $lt: 80 }`.

The output shows the following documents:

```
[{"_id": "6a8d24b77a102a47ae8ce5b1", "rollNo": 112, "name": "Pooja", "age": 20, "department": "IT", "marks": 84}, {"_id": "6a8d24b77a102a47ae8ce5b1", "rollNo": 113, "name": "Ramesh", "age": 22, "department": "ECE", "marks": 79}, {"_id": "6a8d24b77a102a47ae8ce5b1", "rollNo": 101, "name": "John", "age": 20, "department": "ECE", "marks": 75, "city": "Hyderabad"}]
```



The screenshot shows the OneCompiler MongoDB Online Editor interface. The left pane contains a JavaScript script (script.js) with 40 lines of code. The right pane shows the output of the script, which is a JSON array of student documents. The script performs the following operations:

- Inserts a single document: `{ rollNo: 111, name: "Vijay", age: 21, department: "CSE", marks: 86, city: "Hyderabad" }`
- Counts the number of documents in the `students` collection.
- Inserts multiple documents: `{ rollNo: 112, name: "Pooja", age: 20, department: "IT", marks: 84 }, { rollNo: 113, name: "Ramesh", age: 22, department: "ECE", marks: 79 }`
- Counts the number of documents in the `students` collection.
- Finds all documents in the `students` collection.
- Finds a document by `rollNo: 101`.
- Finds a document by `name: "John"`.
- Counts the number of documents in the `students` collection.
- Counts the number of documents in the `students` collection where `department: "CSE"`.
- Finds documents where `marks: { $gt: 80 }`.
- Finds documents where `marks: { $gte: 80 }`.
- Finds documents where `marks: { $lt: 80 }`.

The output shows the following documents:

```
[{"_id": "6a8d24b77a102a47ae8ce5b8", "rollNo": 110, "name": "Meena", "age": 20, "department": "AI&DS", "marks": 93, "city": "Hyderabad"}, {"_id": "6a8d24b77a102a47ae8ce5b9", "rollNo": 111, "name": "Vijay", "age": 21, "department": "CSE", "marks": 86, "city": "Hyderabad"}, {"_id": "6a8d24b77a102a47ae8ce5ba", "rollNo": 112, "name": "Pooja", "age": 20, "department": "IT", "marks": 84, "city": "Hyderabad"}]
```

MongoDB Commands

R.Radhakrishna

2520090063

The screenshot displays the OneCompiler MongoDB Online Editor interface. The left pane shows a JavaScript script for interacting with a MongoDB database. The script performs the following operations:

- Inserts a document into the `students` collection: `{ rollNo: 111, name: "Vijay", age: 21, department: "CSE", marks: 86, city: "Hyderabad" }`.
- Counts the documents in the `students` collection.
- Inserts multiple documents into the `students` collection: `{ rollNo: 112, name: "Pooja", age: 20, department: "IT", marks: 84 }, { rollNo: 113, name: "Ramesh", age: 22, department: "ECE", marks: 79 }`.
- Counts the documents in the `students` collection.
- Finds all documents in the `students` collection and prints them as JSON.
- Finds a document with `rollNo: 101`.
- Finds a document with `name: "John"`.
- Counts the documents in the `students` collection.
- Counts the documents in the `students` collection where the department is "CSE".
- Finds documents where marks are greater than 80.
- Finds documents where marks are greater than or equal to 80.
- Finds documents where marks are less than 80.

The right pane shows the output of the script, which is a JSON array of documents:

```
[{"_id": "ObjectId('6a8d24b77a102a47ae8ce5b7')", "rollNo": 108, "name": "Sita", "age": 21, "department": "CSE", "marks": 91, "city": "Hyderabad"}, {"_id": "ObjectId('6a8d24b77a102a47ae8ce5b8')", "rollNo": 109, "name": "Arun", "age": 22, "department": "AI&DS", "marks": 87, "city": "Kurnool"}, {"_id": "ObjectId('6a8d24b77a102a47ae8ce5b9')", "rollNo": 110, "name": "Vijay", "age": 21, "department": "CSE", "marks": 86, "city": "Hyderabad"}, {"_id": "ObjectId('6a8d24b77a102a47ae8ce5ba')", "rollNo": 111, "name": "Pooja", "age": 20, "department": "IT", "marks": 84, "city": "Hyderabad"}, {"_id": "ObjectId('6a8d24b77a102a47ae8ce5bb')", "rollNo": 112, "name": "Ramesh", "age": 22, "department": "ECE", "marks": 79, "city": "Hyderabad"}]
```

MongoDB Commands

R.Radhakrishna

2520090063

The screenshot displays the OneCompiler MongoDB Online Editor interface. The left pane shows a JavaScript script for interacting with a MongoDB database. The script includes commands for inserting documents, counting documents, and finding documents based on various criteria. The right pane shows the output of the script, which is a JSON array of documents. The output includes documents for 'Vijay', 'Pooja', 'Ramesh', and 'John'.

```
14 })
15 db.students.find()
16 db.students.find().forEach(printjson)
17 db.students.insertOne({
18   rollNo: 111,
19   name: "Vijay",
20   age: 21,
21   department: "CSE",
22   marks: 86,
23   city: "Hyderabad"
24 })
25 db.students.countDocuments()
26 db.students.insertMany([
27   { rollNo: 112, name: "Pooja", age: 20, department: "IT", marks: 84 },
28   { rollNo: 113, name: "Ramesh", age: 22, department: "ECE", marks: 79 }
29 ])
30 db.students.countDocuments()
31 db.students.find()
32 db.students.findOne()
33 db.students.findOne({ rollNo: 101 })
34 db.students.find({ name: "John" })
35 db.students.countDocuments()
36 db.students.countDocuments({ department: "CSE" })
37 db.students.find({ marks: { $gt: 80 } })
38 db.students.find({ marks: { $gte: 80 } })
39 db.students.find({ marks: { $lt: 80 } })
40
```

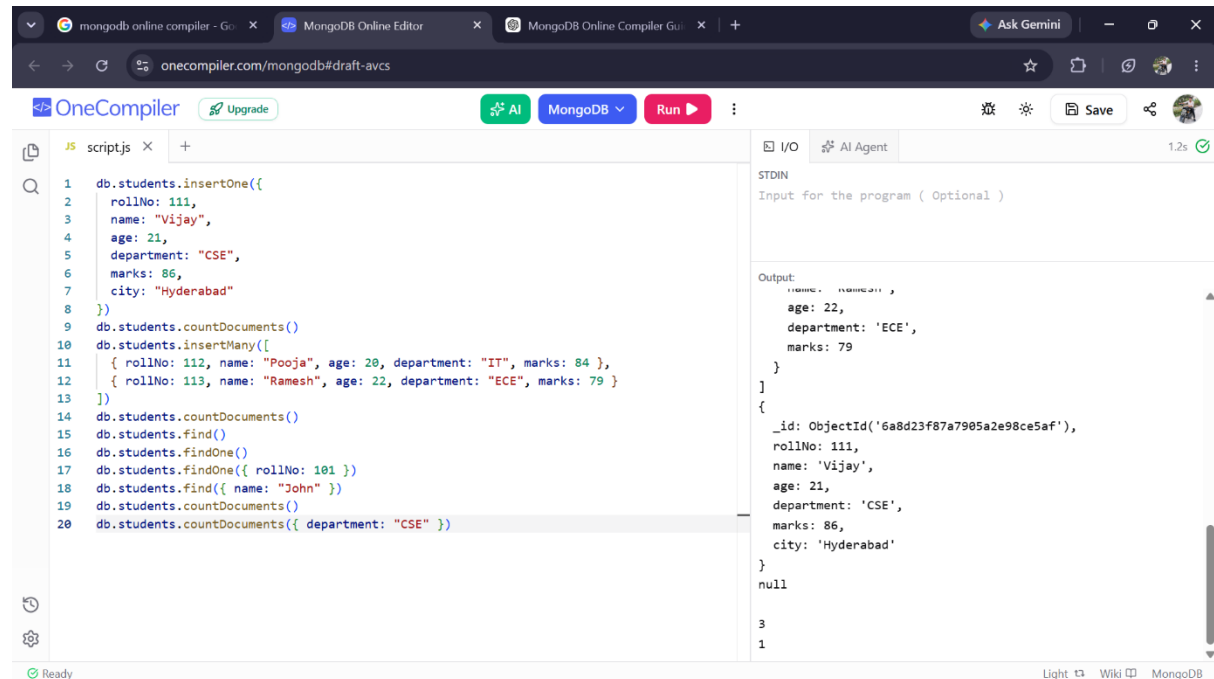
Output:

```
{
  "_id": ObjectId('6a8d24b77a102a47ae8ce5b1'),
  "rollNo": 112,
  "name": "Pooja",
  "age": 20,
  "department": "IT",
  "marks": 84
},
{
  "_id": ObjectId('6a8d24b77a102a47ae8ce5b1'),
  "rollNo": 103,
  "name": "Rahul",
  "age": 20,
  "department": "ECE",
  "marks": 75,
  "city": "Hyderabad"
},
{
  "_id": ObjectId('6a8d24b77a102a47ae8ce5b8'),
  "rollNo": 110,
  "name": "Meena",
  "age": 20,
  "department": "AI&DS",
  "marks": 93,
  "city": "Hyderabad"
},
{
  "_id": ObjectId('6a8d24b77a102a47ae8ce5b9'),
  "rollNo": 111,
  "name": "Vijay",
  "age": 21,
  "department": "CSE",
  "marks": 86,
  "city": "Hyderabad"
},
{
  "_id": ObjectId('6a8d24b77a102a47ae8ce5b1'),
  "rollNo": 112,
  "name": "Pooja",
  "age": 20,
  "department": "IT",
  "marks": 84
}
```

MongoDB Commands

R.Radhakrishna

2520090063

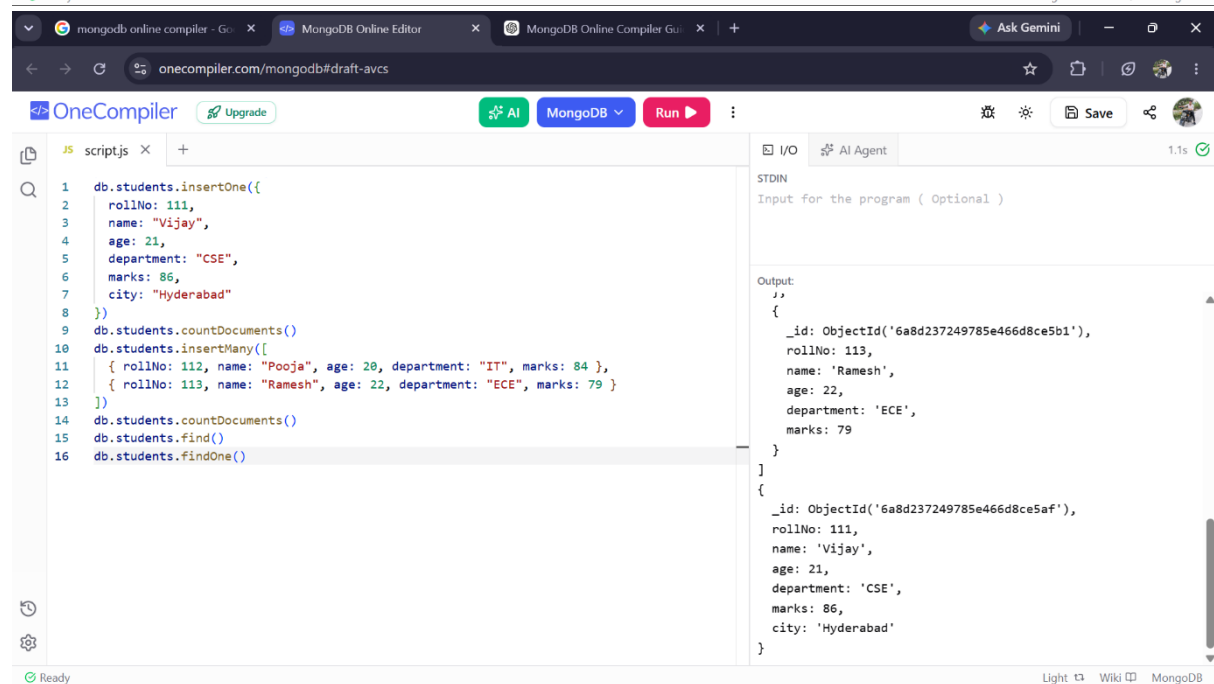


The screenshot shows the OneCompiler MongoDB Online Editor interface. The left pane contains a JavaScript script for interacting with a MongoDB database. The right pane shows the output of the script, which includes the results of insert, count, and find operations.

```
1 db.students.insertOne({
2   rollNo: 111,
3   name: "Vijay",
4   age: 21,
5   department: "CSE",
6   marks: 86,
7   city: "Hyderabad"
8 })
9 db.students.countDocuments()
10 db.students.insertMany([
11   { rollNo: 112, name: "Pooja", age: 20, department: "IT", marks: 84 },
12   { rollNo: 113, name: "Ramesh", age: 22, department: "ECE", marks: 79 }
13 ])
14 db.students.countDocuments()
15 db.students.find()
16 db.students.findOne()
17 db.students.findOne({ rollNo: 101 })
18 db.students.find({ name: "John" })
19 db.students.countDocuments()
20 db.students.countDocuments({ department: "CSE" })
```

Output:

```
{
  age: 22,
  department: 'ECE',
  marks: 79
}
{
  _id: ObjectId('6a8d23f87a7905a2e98ce5af'),
  rollNo: 111,
  name: 'Vijay',
  age: 21,
  department: 'CSE',
  marks: 86,
  city: 'Hyderabad'
}
null
3
1
```



The screenshot shows the OneCompiler MongoDB Online Editor interface. The left pane contains a JavaScript script for interacting with a MongoDB database. The right pane shows the output of the script, which includes the results of insert, count, and find operations.

```
1 db.students.insertOne({
2   rollNo: 111,
3   name: "Vijay",
4   age: 21,
5   department: "CSE",
6   marks: 86,
7   city: "Hyderabad"
8 })
9 db.students.countDocuments()
10 db.students.insertMany([
11   { rollNo: 112, name: "Pooja", age: 20, department: "IT", marks: 84 },
12   { rollNo: 113, name: "Ramesh", age: 22, department: "ECE", marks: 79 }
13 ])
14 db.students.countDocuments()
15 db.students.find()
16 db.students.findOne()
```

Output:

```
{
  _id: ObjectId('6a8d237249785e466d8ce5b1'),
  rollNo: 113,
  name: 'Ramesh',
  age: 22,
  department: 'ECE',
  marks: 79
}
{
  _id: ObjectId('6a8d237249785e466d8ce5af'),
  rollNo: 111,
  name: 'Vijay',
  age: 21,
  department: 'CSE',
  marks: 86,
  city: 'Hyderabad'
}
```

MongoDB Commands

R.Radhakrishna

2520090063

Ask Gemini

—

×

☆

⚙️

☀️

Save

I/O

AI Agent

1.1s ✓

ITDIN

Input for the program (Optional)

Output:

```
City: Hyderabad
},
{
  _id: ObjectId('6a8d22fc4fcd0c4fd68ce5b0'),
  rollNo: 112,
  name: 'Pooja',
  age: 20,
  department: 'IT',
  marks: 84
},
{
  _id: ObjectId('6a8d22fc4fcd0c4fd68ce5b1'),
  rollNo: 113,
  name: 'Ramesh',
  age: 22,
  department: 'ECE',
  marks: 79
}
|
```

Light ↺ Wiki 📖 MongoDB

MongoDB Commands

R.Radhakrishna

2520090063

Ask Gemini

—

×

☆

🔧

☀

Save

I/O

AI Agent

1.1s ✓

STDIN

Input for the program (Optional)

Output:

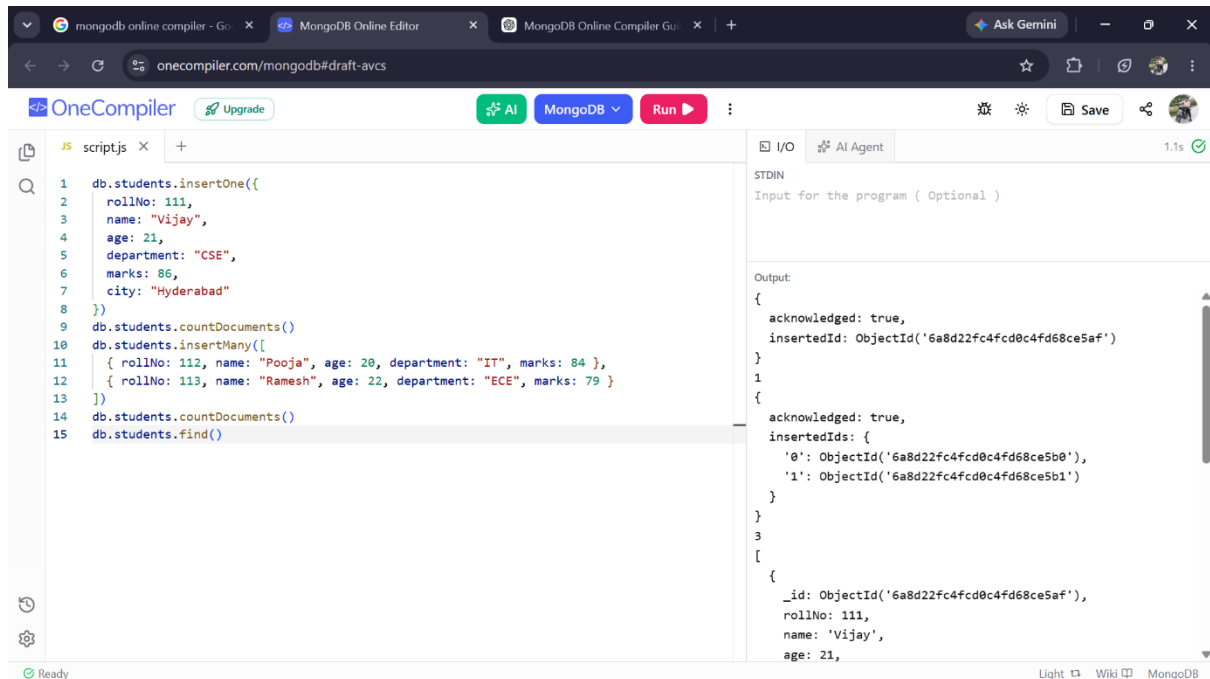
```
{
  _id: ObjectId('6a8d22fc4fcd0c4fd68ce5af'),
  rollNo: 111,
  name: 'Vijay',
  age: 21,
  department: 'CSE',
  marks: 86,
  city: 'Hyderabad'
},
{
  _id: ObjectId('6a8d22fc4fcd0c4fd68ce5b0'),
  rollNo: 112,
  name: 'Pooja',
  age: 20,
  department: 'IT',
  marks: 84
},
{
  _id: ObjectId('6a8d22fc4fcd0c4fd68ce5b1'),
  rollNo: 113,
  name: 'Ravi',
  age: 22,
  department: 'CSE',
  marks: 88,
  city: 'Hyderabad'
}
```

Light 🔁 Wiki 📖 MongoDB

MongoDB Commands

R.Radhakrishna

2520090063



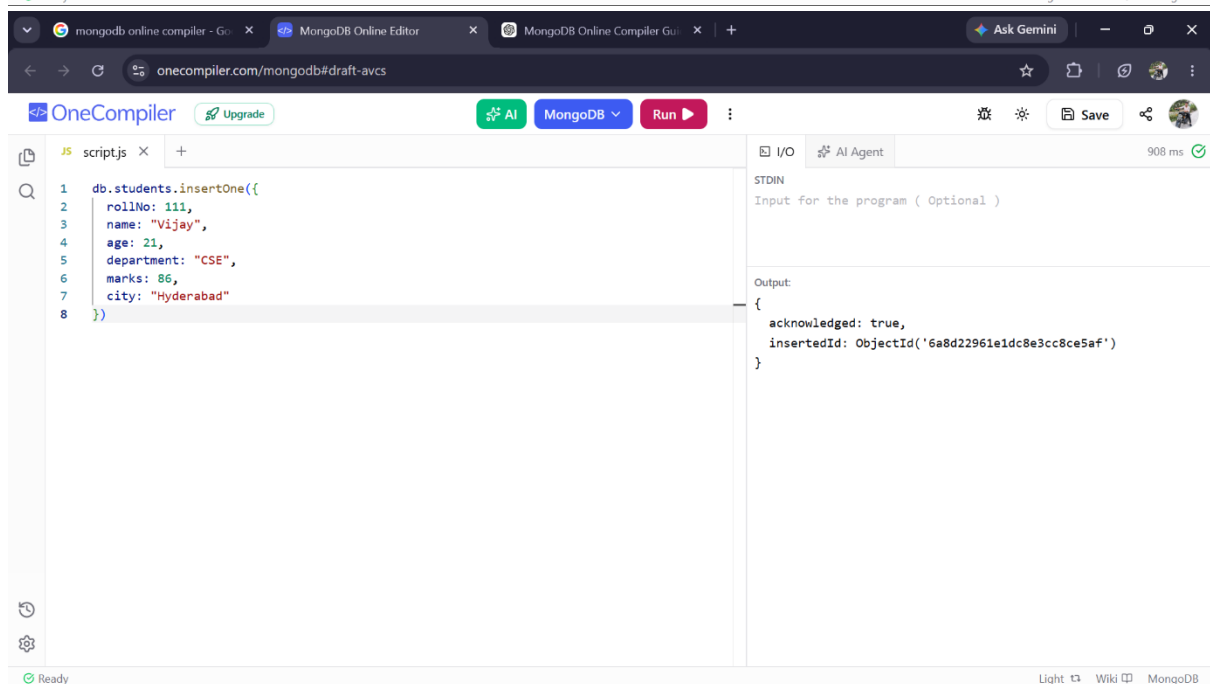
The screenshot shows the OneCompiler MongoDB Online Editor interface. The code editor on the left contains the following JavaScript code:

```
1 db.students.insertOne({
2   rollNo: 111,
3   name: "Vijay",
4   age: 21,
5   department: "CSE",
6   marks: 86,
7   city: "Hyderabad"
8 })
9 db.students.countDocuments()
10 db.students.insertMany([
11   { rollNo: 112, name: "Pooja", age: 20, department: "IT", marks: 84 },
12   { rollNo: 113, name: "Ramesh", age: 22, department: "ECE", marks: 79 }
13 ])
14 db.students.countDocuments()
15 db.students.find()
```

The right panel shows the output of the first command, `db.students.insertOne()`:

```
{
  acknowledged: true,
  insertedId: ObjectId('6a8d22fc4fcd0c4fd68ce5af')
}
```

The status bar at the bottom indicates the execution time was 1.1s.



The screenshot shows the OneCompiler MongoDB Online Editor interface. The code editor on the left contains the following JavaScript code:

```
1 db.students.insertOne({
2   rollNo: 111,
3   name: "Vijay",
4   age: 21,
5   department: "CSE",
6   marks: 86,
7   city: "Hyderabad"
8 })
```

The right panel shows the output of the `db.students.insertOne()` command:

```
{
  acknowledged: true,
  insertedId: ObjectId('6a8d22961e1dc8e3cc8ce5af')
}
```

The status bar at the bottom indicates the execution time was 908 ms.

MongoDB Commands

R.Radhakrishna

2520090063

I/O

AI Agent

1.7s 

STDIN

Input for the program (Optional)

Output:

```
{
  _id: ObjectId('6a8d221c676b5f4d9f8ce5b7'),
  rollNo: 109,
  name: 'Arun',
  age: 22,
  department: 'AI&DS',
  marks: 87,
  city: 'Kurnool'
}
{
  _id: ObjectId('6a8d221c676b5f4d9f8ce5b8'),
  rollNo: 110,
  name: 'Meena',
  age: 20,
  department: 'AI&DS',
  marks: 93,
  city: 'Hyderabad'
}
```

Light  Wiki  MongoDB

MongoDB Commands

R.Radhakrishna

2520090063

I/O

AI Agent

1.7s 

STDIN

Input for the program (Optional)

Output:

```
    age: 20,
    department: 'ECE',
    marks: 75,
    city: 'Hyderabad'
}
{
  _id: ObjectId('6a8d221c676b5f4d9f8ce5b2'),
  rollNo: 104,
  name: 'Priya',
  age: 22,
  department: 'IT',
  marks: 88,
  city: 'Warangal'
}
{
  _id: ObjectId('6a8d221c676b5f4d9f8ce5b3'),
  rollNo: 105,
  name: 'Kiran',
  age: 21,
```

Light  Wiki  MongoDB



Save



I/O

AI Agent

1.7s



STDIN

Input for the program (Optional)

Output:

```
{
  _id: ObjectId('6a8d221c676b5f4d9f8ce5b6'),
  rollNo: 108,
  name: 'Sita',
  age: 21,
  department: 'CSE',
  marks: 91,
  city: 'Hyderabad'
},
{
  _id: ObjectId('6a8d221c676b5f4d9f8ce5b7'),
  rollNo: 109,
  name: 'Arun',
  age: 22,
  department: 'AI&DS',
  marks: 87,
  city: 'Kurnool'
},
{
```

Light

Wiki

MongoDB

MongoDB Commands

R.Radhakrishna

2520090063

Output:

```
-----, ----- ,
  rollNo: 105,
  name: 'Kiran',
  age: 21,
  department: 'CSE',
  marks: 95,
  city: 'Hyderabad'
},
{
  _id: ObjectId('6a8d221c676b5f4d9f8ce5b4'),
  rollNo: 106,
  name: 'Anu',
  age: 20,
  department: 'IT',
  marks: 82,
  city: 'Guntur'
},
{
  _id: ObjectId('6a8d221c676b5f4d9f8ce5b5'),
  rollNo: 107,
```

MongoDB Commands

R.Radhakrishna

2520090063

I/O

AI Agent

1.7s 

STDIN

Input for the program (Optional)

Output:

```
_id: ObjectId('6a8d221c676b5f4d9f8ce5b1'),
rollNo: 103,
name: 'Rahul',
age: 20,
department: 'ECE',
marks: 75,
city: 'Hyderabad'
},
{
  _id: ObjectId('6a8d221c676b5f4d9f8ce5b2'),
  rollNo: 104,
  name: 'Priya',
  age: 22,
  department: 'IT',
  marks: 88,
  city: 'Warangal'
},
{
  _id: ObjectId('6a8d221c676b5f4d9f8ce5b3'),
```

Light  Wiki  MongoDB

MongoDB Commands

R.Radhakrishna

2520090063

I/O

AI Agent

1.7s 

STDIN

Input for the program (Optional)

Output:

```
[
  {
    _id: ObjectId('6a8d221c676b5f4d9f8ce5af'),
    rollNo: 101,
    name: 'John',
    age: 20,
    department: 'CSE',
    marks: 85,
    city: 'Hyderabad'
  },
  {
    _id: ObjectId('6a8d221c676b5f4d9f8ce5b0'),
    rollNo: 102,
    name: 'Alice',
    age: 21,
    department: 'CSE',
    marks: 90,
    city: 'Vijayawada'
  }
]
```

Light  Wiki  MongoDB

MongoDB Commands

R.Radhakrishna

2520090063

The image displays two screenshots of the OneCompiler MongoDB Online Editor interface, demonstrating database operations.

Top Screenshot:

- Code Editor:** A JavaScript script is shown with the following commands:

```
1 db.createCollection("students")
2 show collections
3 db.students.insertMany([
4   { rollNo: 101, name: "John", age: 20, department: "CSE", marks: 85, city: "Hyderabad" },
5   { rollNo: 102, name: "Alice", age: 21, department: "CSE", marks: 90, city: "Vijayawada" },
6   { rollNo: 103, name: "Rahul", age: 20, department: "ECE", marks: 75, city: "Hyderabad" },
7   { rollNo: 104, name: "Priya", age: 22, department: "IT", marks: 88, city: "Warangal" },
8   { rollNo: 105, name: "Kiran", age: 21, department: "ECE", marks: 85, city: "Hyderabad" },
9   { rollNo: 106, name: "Anu", age: 20, department: "AI&DS", marks: 92, city: "Guntur" },
10  { rollNo: 107, name: "Ravi", age: 23, department: "ECE", marks: 68, city: "Nellore" },
11  { rollNo: 108, name: "Sita", age: 21, department: "CSE", marks: 91, city: "Hyderabad" },
12  { rollNo: 109, name: "Arjun", age: 22, department: "AI&DS", marks: 87, city: "Kurnool" },
13  { rollNo: 110, name: "Meena", age: 20, department: "AI&DS", marks: 93, city: "Hyderabad" }
14 ])
15 db.students.find()
16 db.students.find().forEach(printjson)
```
- Output Panel:** Shows the result of the `show collections` command: `{ ok: 1 }`. Below it, the output of `db.students.insertMany()` is displayed, showing `acknowledged: true` and a list of `insertedIds` for each document.

Bottom Screenshot:

- Code Editor:** A JavaScript script is shown with the following commands:

```
1 use studentDB
2 db
3 show dbs
4
```
- Output Panel:** Shows the result of the `use studentDB` command: `switched to db studentDB`. Below it, the output of `show dbs` is displayed, showing `studentDB`.